

Context-Based Meta-Reinforcement Learning for Rapid Adaptation in Multi-Track Racing

Felix Nathaniel Surjodinoto*, Keandre Rafael*, Delroy*, Samuel Philip*

*Computer Science

Bina Nusantara University

Jakarta, Indonesia

{felix.surjodinoto, keandre.rafael, delroy, samuel.philip}@binus.ac.id

Abstract—

The current state-of-the-art reinforcement learning algorithm shows that rapid adaptation remains one of the biggest hurdles in RL breakthroughs, especially in an environment where speed and high responsiveness are important. Our paper attempts to prove that a sample efficiency framework is possible without compromising any results in consumer-grade hardware by introducing a hybrid architecture from Dropout Q-Functions (DroQ) and Soft Actor-Critic (SAC) to gain better generalization learning throughout the task, which will be augmented by a Gated Recurrent Unit (GRU) to capture short-term memorization. Furthermore, to improve learning latency, we add Prioritized Experience Replay (PER) and Behavior Cloning (BC). After several experiments in Trackmania 2020, the current framework demonstrates that our method reduces adaptation time from several days to just 4 hours on a GTX 1650 laptop, achieving competitive lap times with significantly lower computational overhead than standard baselines.

Index Terms—Meta-reinforcement learning, multi-task learning, autonomous racing, rapid adaptation, RNN, SAC, REDQ, RESeL.

I. INTRODUCTION

In the world of autonomous racing, adaptation is undoubtedly one of the most important aspects in artificial intelligence. An experienced racing driver must constantly adapt to the changing circumstances during competition, either anticipating and adjusting track patterns or conditions and responding to competitor behavior because its complexity, highly nonlinear, and requires real-time decision-making at high speeds under unpredictable conditions [1]. Unlike normal AVs, autonomous racing pushes vehicles to their limits to minimize lap times. This requires control policies to make those decisions at exceptionally high frequencies (10 Hz - 100Hz) [2]–[4].

To accelerate development in this area, the community relies on environments such as F1TENTH [1], CARLA [5], and other game based platforms like Assetto Corsa [6] that provide standardized hardware and software for racing competitions, enabling safe and rapid experiments. This thesis utilizes the TrackMania racing environment, accessed via the TMRL framework which provides continuous control and low-dimensional feedback around 20hz [7].

In standard RL, the agent would similarly need to interact with the environment via trial-and-error with the environment many times in order to reach human-level performance [8] and that RL is often task-specific **task-specific**: the policies learned

to understand just one single environment, when deployed under **distribution shift** (e.g., unseen tracks or altered dynamics) can fail catastrophically [9], [10]. However Context-based meta-RL methods like PEARL have shown promise in continuous control benchmarks [11] yet have not been evaluated for high response time like racing tasks.

Despite the progress of RL in racing environment, the existing approaches typically train and evaluate on a single track, and their ability to rapidly adapt to unseen tracks with minimal data remains untouched.

We present a novel, resource-efficient framework for rapid adaptation in autonomous racing, explicitly designed to operate on consumer-grade hardware (e.g., NVIDIA GTX 1650). Unlike diverse multi-modal systems that demand high-end workstations, our approach leverages a Hybrid LiDAR-Vision architecture within the TrackMania 2020 environment [7]. We demonstrate that by fusing geometric data (LiDAR) with temporal context (GRU), it is possible to achieve robust, adaptive racing behaviors with a fraction of the computational overhead of traditional methods [12].

Our framework introduces three key contributions to the democratized racing research landscape. First, we propose an efficient Sensing module that integrates sparse LiDAR rays. Second, we implement a Context-Based Meta-RL mechanism using Gated Recurrent Units (GRUs), allowing the agent to infer and adapt to unseen surface friction (e.g., Ice vs. Asphalt) in a zero-shot manner, eliminating the need for gradient updates during deployment [12]. Finally, we establish a stable training protocol using Dropout-Q (DroQ) and Randomized Ensembled Double Q-Learning (REDQ), proving that sample-efficient learning is achievable on a single GPU [13], [14].

We validate our approach through extensive empirical analysis, comparing our hybrid architecture against standard baselines on complex multi-surface tracks. The results demonstrate that our agent not only learns to navigate challenging geometries but typically adapts to friction shifts within seconds, highlighting the potential for accessible, high-performance meta-learning in the field of self-driving racing.

The main contributions of this thesis are as follows:

- 1) We integrate a **GRU-based context encoder** with SAC with dual critics and DroQ regularization (dropout, LayerNorm) for autonomous racing, demonstrating improved sample efficiency on single-track learning.

- 2) We propose **FiLM conditioning** as a mechanism for few-shot adaptation, where the context vector modulates the actor and critic networks via learned scale and shift parameters.
- 3) We design an **online adaptation protocol** that updates only the FiLM generator on a held-out test track, stabilized by a self-imitation regularizer to prevent catastrophic forgetting.
- 4) We provide an **empirical evaluation** on the TrackMania environment (via TMRL), comparing learning curves on a single track and adaptation curves across a multi-track train/test split.

II. RELATED WORKS

Our research occupies the convergence of autonomous racing, meta-reinforcement learning, and sample-efficient off-policy control. This section examines previous methodologies in these fields and situates our contribution within the wider context of adaptive control.

A. Deep Reinforcement Learning for Autonomous Racing

Recurrent neural networks have been widely used in reinforcement learning to address partial observability by capturing temporal dependencies across sequences of observations, actions, and rewards. RNN-based agents are also able to infer latent task information by maintaining a hidden state that summarizes interaction history in Partially Observable Markov Decision Processes (POMDPs), where the full environment state is not directly accessible [15]. This capability is suitable for autonomous driving and racing, where task identity and environment dynamics must be inferred online through experience.

RNNs are commonly integrated with feedforward networks, such as Multi-Layer Perceptrons (MLPs), to form recurrent policies. The RNN component serve as a context encoder, extracting hidden state information, usually includes the current observation, last action, and reward to produce a latent representation of the task. This representation is then used by the MLP layers to generate control actions.

B. Randomized Ensemble Double Q-Learning (REDQ)

REDQ is model-free reinforcement learning algorithm designed to improve sample efficiency in continuous control tasks. REDQ is built upon the SAC framework by maintaining an ensemble of critic networks and randomly sampling a subset of critics for target value estimation during training. The high sample efficiency is important for autonomous racing, where real-world data collection is expensive and limited. The agent is allowed to learn from a smaller number of environmental interactions, making it more effective than Soft Actor-Critic (SAC).

Dropout Q-Functions (DroQ) is a variant of REDQ that reduces the computational burden of REDQ by using a smaller ensemble combined with dropout and layer normalization to achieve double efficiency [13]. Both REDQ and DroQ aim

to enhance sample efficiency, REDQ relies on ensemble-based value estimation, whereas DroQ leverages stochastic regularization and delayed policy updates.

C. Probabilistic Embeddings for Actor-Critic RL (PEARL) for Continuous Control

PEARL is designed for multi-task adaptation problems with high data efficiency. It trains a stochastic context encoder to infer a probabilistic posterior over the latent context variable z based on experience from the current task.

The algorithm is built on top of the SAC framework for its high sample efficiency and entropy-based exploration and it also allows for vast replay buffer of past experiences which improves learning efficiency [16].

D. RESeL: Efficient Recurrent Off-Policy Meta-RL

RESeL is an off-policy Meta-Reinforcement Learning algorithm designed to address training instability in RNN-based context encoders suffers from divergence or poor performance as sequence lengths increase, specifically when trained with the same learning rate as policy networks. RESeL mitigates this issue by utilizing a significantly lower learning rate for context encoder while still having a standard learning rate for the MLP layers of policy. By slowing down updates to the context encoder, RESeL can ensure that the agent's representation remains stable without sacrificing the general speed of the training process [15].

RESeL uses the SAC framework and incorporates the ensemble-Q mechanism from DroQ and utilizes delayed policy update strategy to make sure that the critic is trained long enough before the policy is adjusted.

III. METHODOLOGY

The goal of few-shot meta-reinforcement learning is to train a policy that can quickly adapt to a new task using only a few episodes and training iterations. To accomplish this, the policy is trained during a meta-learning phase on a set of tasks, such that the trained policy can quickly adapt to new tasks using only a small number of interactions. In effect, the meta-learning problem treats entire tasks as training examples [17].

We consider a policy, denoted π_θ , that maps observations s to actions a . During meta-learning, the policy is trained to be able to adapt to a large or infinite number of tasks. Since we would like to apply our framework to autonomous racing, we formalize the task distribution below.

Formally, each task $\mathcal{T}_i = \{\mathcal{L}_i, q_i(s_1), q_i(s_{t+1}|s_t, a_t), H\}$ consists of a loss function $\mathcal{L}_i$, a distribution over initial observations $q_i(s_1)$, a transition distribution $q_i(s_{t+1}|s_t, a_t)$, and an episode length H . In the racing domain, each task corresponds to a distinct track, where the transition dynamics q_i and optimal racing line differ across tracks. The loss $\mathcal{L}_i(s_1, a_1, \dots, s_H, a_H) \rightarrow \mathbb{R}$ provides task-specific feedback in the form of a cost function in a Markov decision process.

In our meta-learning scenario, we consider a distribution over tasks $p(\mathcal{T})$ that we want our policy to be able to adapt

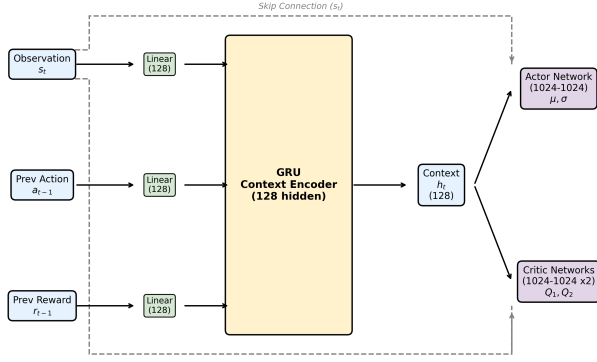


Fig. 1. Architecture overview for gradient-based (MAML-style) adaptation versus context-based adaptation.

to. In the K -shot learning setting, the policy is trained to learn a new task $\mathcal{T}_i$ drawn from $p(\mathcal{T})$ from only K episodes drawn from q_i and feedback $\mathcal{L}_{\mathcal{T}_i}$ generated by $\mathcal{T}_i$. During meta-training, a task $\mathcal{T}_i$ is sampled from $p(\mathcal{T})$, the policy is trained with K samples and feedback from the corresponding loss $\mathcal{L}_{\mathcal{T}_i}$ from $\mathcal{T}_i$, and then tested on new samples from $\mathcal{T}_i$. The policy π_θ is then improved by considering how the *test* error on new data from q_i changes with respect to the parameters. At the end of meta-training, new tasks are sampled from $p(\mathcal{T})$, and meta-performance is measured by the policy’s performance after learning from K samples. Generally, tasks used for meta-testing are held out during meta-training.

A. Context-Based Adaptation

We highlight the effectiveness of the contextual based adaptation approach by comparing it to gradient-based meta-learning techniques within the autonomous racing domain (Figure 1). The target is to train the policies that can swiftly adapt to unseen tracks without requiring detailed gradient updates at test time. We choose to implement a GRU-based context encoder that collects information from past encounters, which allows adaptation via inference instead of optimization. We analyze both single-track learning and multi-track generalization to emphasize the capability of the particular formulations to deal with both scenarios. This model illustrates that context-based meta-RL are able to achieve rapid adaptation to new tasks while gradient-based approaches require costly inner-loop updates. **Gradient-Based Adaptation:** First, we consider the gradient-based meta-learning approach exemplified by MAML [17]. This depicts the adaptation mechanism of agents trained under MAML-style updates. While agents trained under MAML adapt through explicit gradient steps on task-specific data, the context-based formulation adapts through implicit inference via the evolving hidden state.

When formulating the adaptation concerning gradient updates, the agent must store task-specific trajectories and compute inner-loop gradients before acting optimally. This, however, neglects the computational cost and memory requirements of maintaining such gradients during real-time

racing at 20 Hz. Assuming the agent encounters a new track with different dynamics, an agent trained under MAML must first collect experience, compute gradients, and update its parameters before achieving optimal performance. This results in suboptimal behavior during the initial adaptation phase while giving the agent limited time to observe and counteract new track characteristics.

Context-Based Adaptation. Instead, formulating the adaptation concerning context inference allows the agent to observe task characteristics through accumulated experience and compensate for distribution shift without explicit optimization. We define a context vector $h_t \in \mathbb{R}^{128}$ that summarizes the agent’s experience up to time t . This context is computed by a GRU encoder [12] that takes as input the observation s_t , previous action a_{t-1} , and previous reward r_{t-1} :

$$h_t = \text{GRU}(f_\phi(s_t, a_{t-1}, r_{t-1}), h_{t-1}) \quad (1)$$

where f_ϕ denotes pre-encoding layers that project each input modality to a shared embedding space. The context vector h_t is then concatenated with the current observation to condition the policy:

$$a_t \sim \pi_\theta(a|s_t, h_t) \quad (2)$$

This formulation allows the policy to adapt its behavior based on accumulated experience without requiring gradient updates. As the agent interacts with a new track, the context vector evolves to capture task-relevant information such as curvature patterns, friction characteristics, and optimal racing lines. This evolution occurs naturally through forward passes without requiring backpropagation, enabling real-time adaptation during racing.

B. Experimental Setup

We conduct the experiments in TrackMania 2020 using the TMRL framework [7]. The environment provides a low-dimensional sensory interface consisting of 19 LiDAR rays for track boundary detection, vehicle speed, a buffer of the last two actions, and the previous reward. The action space consists of continuous steering and throttle commands in $[-1, 1]$. The environment operates at 20 Hz, requiring control decisions every 50 ms. **Reward Function.** The reward function encourages forward progress while penalizing time:

$$r_t = r_{\text{progress}} + r_{\text{constant}} + r_{\text{completion}} \quad (3)$$

where $r_{\text{constant}} = -0.07$ per step to encourage faster completion, r_{progress} rewards forward movement along the track, and $r_{\text{completion}} = 1000$ for completing the track. **Training Infrastructure.** Training uses a distributed architecture with three components: (1) a *worker* process that runs in TrackMania and collects experience, (2) a *server* process that manages the replay buffer and weight distribution, and (3) a *trainer* process that performs gradient updates on an NVIDIA GTX 1650 GPU with 4GB VRAM. **Hyperparameters.** Table I summarizes the key hyperparameters used in our experiments. We use a batch size of 8192 and train for 2000 gradient steps per epoch. The replay buffer holds 1 million transitions. **Evaluation**

TABLE I
TRAINING HYPERPARAMETERS

Hyperparameter	Value
Discount factor γ	0.995
Polyak coefficient τ	0.995
Actor/Critic learning rate η	3×10^{-4}
Context encoder learning rate η_{CE}	1×10^{-5}
Entropy learning rate	1×10^{-3}
Target entropy $\bar{\mathcal{H}}$	-2.0
Initial entropy coefficient α	0.2
Batch size	4096
Sequence length L	4
Hidden layer sizes	(1024, 1024)
Context hidden size	128
Encoder hidden size	128
Dropout rate	0.01
Policy delay	2
Number of critics N	2
Replay buffer size	1,000,000
Training steps per epoch	2000

Protocol. We evaluate our approach by measuring: (1) sample efficiency on single-track learning, comparing learning curves of cumulative return against training epochs, and (2) episode length as a proxy for stability, where longer episodes indicate the agent can drive further without crashing.

REFERENCES

- [1] B. Hildisch, E. Ghignone, N. Baumann, C. Hu, A. Carron, and M. Magno, “Drive fast, learn faster: On-board rl for high performance autonomous racing,” *ArXiv*, vol. abs/2505.07321, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:278501348>
- [2] W. Xiao, T. He, J. M. Dolan, and G. Shi, “Safe deep policy adaptation,” *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 17 286–17 292, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:264128072>
- [3] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,” *Science Robotics*, vol. 8, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261706234>
- [4] J. Subosits, J. Lee, S. Manuel, P. Tylkin, and A. Balachandran, “Towards an autonomous test driver: High-performance driver modeling via reinforcement learning,” *ArXiv*, vol. abs/2412.03803, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:274515100>
- [5] A. Dosovitskiy, G. Ros, F. Codevilla, A. M. López, and V. Koltun, “Carla: An open urban driving simulator,” in *Conference on Robot Learning*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:5550767>
- [6] A. Remonda, N. Hansen, A. Raji, N. Musiu, M. Bertogna, E. E. Veas, and X. Wang, “A simulation benchmark for autonomous racing with large-scale human data,” *ArXiv*, vol. abs/2407.16680, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271334499>
- [7] “TMRL: Trackmania reinforcement learning framework,” GitHub repository, accessed: 2026-01-27. [Online]. Available: <https://github.com/trackmania-rl/tmrl>
- [8] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, “Soft actor-critic algorithms and applications,” *ArXiv*, vol. abs/1812.05905, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:55703664>
- [9] A. Murdoch, J. C. Schoeman, and H. W. Jordaan, “Partial end-to-end reinforcement learning for robustness against modelling error in autonomous racing,” *ArXiv*, vol. abs/2312.06406, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266163260>
- [10] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot, and F. Nashashibi, “End-to-end race driving with deep reinforcement learning,” *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2070–2075, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:49655106>
- [11] K. Rakelly, A. Zhou, C. Finn, S. Levine, and D. Quillen, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- [12] J. Chung, Çağlar Gülçehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *ArXiv*, vol. abs/1412.3555, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:5201925>
- [13] T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka, “Dropout q-functions for doubly efficient reinforcement learning,” *ArXiv*, 2025.
- [14] X. Chen, C. Wang, Z. Zhou, and K. Ross, “Randomized ensembled double q-learning: Learning fast without a model,” *ArXiv*, 2021.
- [15] F. Luo, Z. Tu, Z. Huang, and Y. Yu, “Efficient recurrent off-policy rl requires a context-encoder-specific learning rate,” *ArXiv*, vol. abs/2405.15384, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:270045647>
- [16] L. Wen, S. Zhang, H. E. Tseng, B. Singh, D. Filev, and H. Peng, “Improved robustness and safety for pre-adaptation of meta reinforcement learning with prior regularization,” *ArXiv*, 2023.
- [17] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International Conference on Machine Learning (ICML)*, 2017, pp. 1126–1135.